CHAPTER 5: IMPLEMENTATION

This chapter describes the implementation of FactCheck AI (project name: nextn), a Next.js 15 web application that performs multi-modal factual verification using Google Genkit, the Gemini family of models, and Firebase (authentication and Firestore). The user interface follows a deliberate “Twilight Forest” design language with Tailwind CSS and ShadCN-style primitives. Below, each major module is described with its responsibilities, key entry points drawn from the codebase, and representative pseudocode aligned with the actual control flow in the repository.

5.1 Module: Genkit AI Core and Model Configuration

The central AI runtime is configured in src/ai/genkit.ts. A singleton Genkit instance is created with the @genkit-ai/google-genai plugin and a default text model googleai/gemini-3.1-flash-lite-preview. All server-side flows import this shared ai object so prompts, schemas, and flows remain consistent across Tier-1 and Tier-2 verification.

Key artifact:

• ai — Genkit instance exported for use by definePrompt and defineFlow across src/ai/flows/.

```
FUNCTION configure_genkit():
    RETURN genkit({
        plugins: [googleAI()],
        model: 'googleai/gemini-3.1-flash-lite-preview'
    })
```

5.2 Module: Tier-1 Text Verification (Selected Text)

Implemented in src/ai/flows/verify-selected-text-accuracy.ts as a server module ('use server'). The flow verifySelectedTextAccuracy accepts user-selected plain text, invokes a structured prompt, and returns a Zod-validated object: verdict (Likely Accurate | Needs Verification | Potentially Misleading), suggestedCorrectionOrContext, reasoning, an array of sources with reliability (High | Medium | Mixed), and recommendedSearchQuery for follow-up human verification.

Key functions:

• verifySelectedTextAccuracy(input) — Public async entry; delegates to verifySelectedTextAccuracyFlow.
• verifySelectedTextAccuracyPrompt — definePrompt binding input/output schemas to the fact-checker system prompt.
• verifySelectedTextAccuracyFlow — defineFlow implementation; calls the prompt and throws if output is missing.

```
FUNCTION verifySelectedTextAccuracyFlow(selectedText):
    output = await verifySelectedTextAccuracyPrompt({ selectedText })
    IF output IS NULL:
        THROW Error('No output received from the fact-checking prompt.')
    RETURN output
```

5.3 Module: Tier-1 Image Verification (Claims in Visual Media)

Implemented in src/ai/flows/verify-image-accuracy.ts. The input schema requires imageDataUri: a data URI with MIME type and Base64 payload. The multimodal prompt references the image via {{media url=imageDataUri}}. The model extracts the primary factual claim (extractedText), then applies the same verdict taxonomy and source structure as the text flow so the UI can treat both modalities uniformly after the first tier.

Key function:

• verifyImageAccuracy(input) — Runs verifyImageAccuracyFlow; surfaces extraction plus verification in one response.

```
FUNCTION verifyImageAccuracyFlow(imageDataUri):
    output = await verifyImageAccuracyPrompt({ imageDataUri })
    IF output IS NULL:
        THROW Error('No output received from the image fact-checking prompt.')
    RETURN output  // includes extractedText, verdict, sources, recommendedSearchQuery
```

5.4 Module: Tier-2 Deep Scan (Semantic Nuance and Cross-References)

Implemented in src/ai/flows/deep-analysis-flow.ts. The function initiateDeepAnalysis accepts originalText and the initialVerdict from Tier-1. The structured output contains nuanceAnalysis (gray areas and misconceptions), crossReferences (title, url, context per source), and confidenceScore (0–100) representing confidence in the full verification chain.

```
FUNCTION initiateDeepAnalysis(originalText, initialVerdict):
    output = await deepAnalysisPrompt({ originalText, initialVerdict })
    IF output IS NULL:
        THROW Error('Deep analysis failed.')
    RETURN output
```

5.5 Module: Authoritative Text-to-Speech (Truth Audio)

Implemented in src/ai/flows/text-to-speech-flow.ts. generateTruthAudio uses ai.generate with googleAI.model('gemini-2.5-flash-preview-tts'), responseModalities ['AUDIO'], and a prebuilt voice (e.g. Algenib). The flow packages returned media as a base64 data URI for client playback. The wav package is used in the same module for audio processing as implemented in the project.

5.6 Module: Client Verifier Orchestration and Persistence

The primary UI orchestrator is src/components/fact-check/text-verifier.tsx (client component). On mount, it ensures an anonymous Firebase user via initiateAnonymousSignIn when no user exists. handleVerify clears prior results, branches on selectedImage: if an image is present it calls verifyImageAccuracy({ imageDataUri }); otherwise verifySelectedTextAccuracy({ selectedText }). On success, it updates local state and, when user and firestore are available, writes a document to Firestore at path users/{uid}/verificationResults/{verificationId} using setDocumentNonBlocking with fields id, originalText (extracted text for images), verdict, suggestedCorrection, reasoning, sources, and checkedAt (ISO timestamp). handleDeepVerify calls initiateDeepAnalysis with originalText from extractedText fallback and the current Tier-1 verdict.

```
FUNCTION handleVerify():
    IF NOT inputText AND NOT selectedImage: RETURN
    IF selectedImage:
        output = await verifyImageAccuracy({ imageDataUri: selectedImage })
        historyText = output.extractedText
    ELSE:
        output = await verifySelectedTextAccuracy({ selectedText: inputText })
        historyText = inputText
    persist_to_firestore_if_authenticated(historyText, output)

FUNCTION handleDeepVerify():
    text = result.extractedText OR inputText
    RETURN await initiateDeepAnalysis({
        originalText: text,
        initialVerdict: result.verdict
    })
```

5.7 Module: Presentation Layer (Next.js App Router)

Routes are implemented under src/app/. The landing experience src/app/page.tsx composes the marketing hero, feature narrative, and the TextVerifier component; navigation links to /history and /extension-demo. src/app/history/page.tsx hosts the "Intelligence Archive" and renders VerificationHistory from src/components/fact-check/history.tsx, which consumes Firestore collections for the signed-in user. src/app/extension-demo/page.tsx demonstrates highlight-to-verify behavior using MockBrowser from src/components/fact-check/mock-browser.tsx. Global layout and styling are defined in src/app/layout.tsx with Tailwind and shared UI primitives under src/components/ui/.

5.8 Module: Firebase Client Provider and Error Handling

Firebase initialization and React context live under src/firebase/ (e.g. client-provider.tsx, provider.tsx, config.ts). Non-blocking Firestore writes and auth patterns are encapsulated in non-blocking-login.tsx, non-blocking-updates.tsx, and hooks such as use-collection.tsx / use-doc.tsx. FirebaseErrorListener surfaces errors to the UI layer as wired in the application shell.

5.9 Build, Run, and Environment

package.json defines npm run dev as next dev --turbopack -p 9002, npm run build as production next build (note: on Windows, developers may need a cross-env wrapper for NODE_ENV if issues arise), and Genkit dev entry points genkit:dev / genkit:watch targeting src/ai/dev.ts. Environment variables for API keys and Firebase are expected via .env as loaded by Next.js.

This document was generated to mirror the structure of a typical "Chapter 5: Implementation" technical report, with section numbering and pseudocode aligned to the FactCheck AI codebase in the Final Project repository.